A Dynamic Multi-Objective Cost Function for Personalized and Accessible Urban Pathfinding

Aayam Bajaj,
Information Technology,
Bharati Vidyapeeth College of Engineering, Navi-Mumbai, India
aayambajaj4@gmail.com

Atharva Dhokte,
Information Technology,
Bharati Vidyapeeth College of Engineering, Navi-Mumbai, India
atharva.a.d2004@gmail.com

Yuvrajsing Bahure
Information Technology,
Bharati Vidyapeeth College of Engineering, Navi-Mumbai, India
bahureyuvi@gmail.com

Prof. Hitendra Chavan
Department of Information Technology,
Bharati Vidyapeeth College of Engineering, Navi-Mumbai, India
hitendra.chavan@bharatividyapeeth.edu

***Abstract*:** Standard urban navigation systems often fail to meet the needs of individuals with mobility impairments. These systems focus on optimizing the shortest path, ignoring critical accessibility factors like surface quality, dynamic real-time obstacles, and sidewalk slopes, thereby creating an informational gap for users. In order to address these shortcomings, this paper introduces a pathfinding framework, by enhancing the A* search algorithm using a dynamic, multi-objective cost function. With calculation of the traversal cost for every path segment, our methodology provides personalized and context-aware routing. The derivation of this cost happens through the integration of multiple layers of data, including: (1) Personalized user profiles, (2) Community-sourced data on obstructions and temporary obstacles, (3) Machine learning model for predictive risk assessment, and (4) Live weather updates. This results in generation of routes that consider user safety and comfort instead of just distance. Hence this system offers a reliable and accessible navigation experience.

Keywords: Pathfinding, A* Algorithm, Accessibility, Multi-Objective Optimization, Urban Navigation.

I. INTRODUCTION

With the growing population and increase of urban environment spaces, effective navigation is important for social connectivity and freedom of mobility. However, this basic navigation becomes a challenge for those with limited mobility. This includes the surface conditions, construction blockages, slope of sidewalks, and presence of curb ramps, etc. These barriers pose health risks and can also lead to detachment from the community life as well as reduced participation.

Current mapping services provide efficient routing for general pedestrians. Although, they fall short in meeting the mobility needs of users. This is because they only focus on a single objective, like finding the shortest path. This leaves out metrics like comfort, accessibility, and safety. There is data on accessibility, but it is variegated across parts of datasets, which creates a gap for the users who seek a dynamic nature of information. **Fig. 1** provides a clear snapshot of how accessibility can vary across a city.

In order to overcome these challenges, this paper introduces a framework which extends the A* algorithm with a dynamic, multi-objected cost function. We intend to provide personalized routing by integrating multiple layers of information into a cost metric. Our objective is to formulate a cost function that calculates path based on four key aspects:

1. Personalized user-profile with mobility constraints and user preferences.
2. Real-time reporting and community sourced data on temporary obstacles.
3. Risk assessment based on observed patterns using machine learning.
4. Weather-condition data that may affect path metrics.

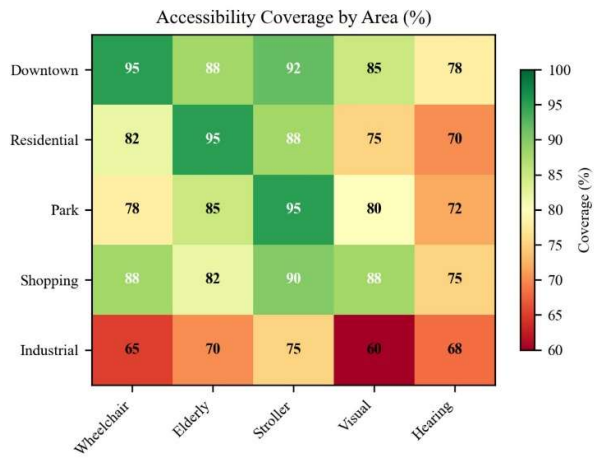

Fig 1. Heatmap of accessibility coverage scores (%) by user profile and urban area type.

II. SYSTEM ARCHITECTURE

The system operates as a computational pipeline designed to transform raw geographic data into a personalized, safe, and accessible routing service. The workflow consists of five stages, from initial data processing to the final path computation.

Step 1: Graph Construction from OpenStreetMap (OSM)
The foundation of our system is a routable graph built from OSM data. The first step is uploading the raw OSM data for a particular region in the procedure. Only "ways" and "nodes" that are relevant to pedestrians designated by tags such as highway='footway' and sidewalk='yes', are chosen by the system after filtering this data. After that, a mathematical graph is produced, in which the nodes stand for intersections and the edges for the segments of paths that connect them.

Each edge is initially stored with its static attributes from OSM, such as its physical length (L(e)) and surface type.

Step 2: Route Request Initialization
This stage begins when a user requests a route.
The backend receives the user's request, which contains three key pieces of information: a **start location**, an **end location**, and a **mobility profile** (containing the necessary metric values like maximum slope, disliked surfaces, etc.). The system then loads the relevant section of the pre-processed graph into the memory to begin dynamic analysis.

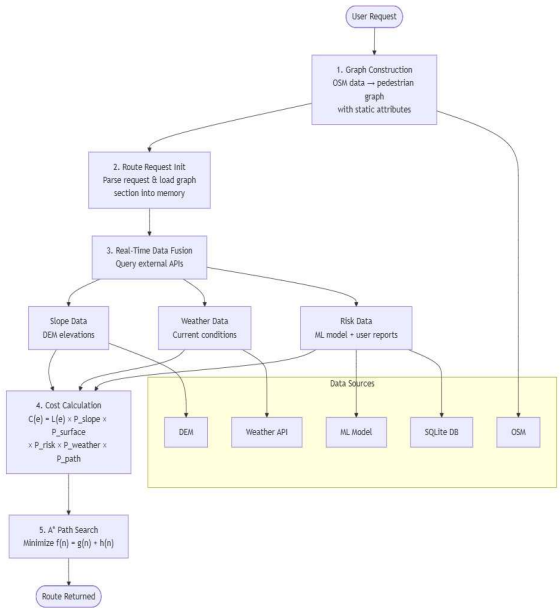

Fig 2. System Architecture

Step 3: Real-Time Data Fusion
With the base graph loaded, the system enhances it with live, dynamic data.

- The system gathers real-time information from multiple sources, for each edge in the graph that is relevant to the potential route
- **Slope Data**: For an accurate slope calculation, it queries a **Digital Elevation Model (DEM)** service to get the elevation of the start and end nodes of the edge.
- **Weather Data**: An API call is made to a live **weather service** to determine current conditions (e.g., rain), which make certain surfaces more hazardous.
- **Risk Data**: It checks **SQLite database** for any user-reported temporary blockages and then queries our internal **Machine Learning (ML) model** for a predicted obstacle probability.

Step 4: Personalized Cost Calculation
This is where the graph becomes fully personalized to the user and the current environment. The system iterates through the edges of the dynamically enhanced graph. For each edge, it calculates a final traversal cost using the multiplicative penalty function. The penalties for 'slope' and 'surface' are calculated specifically based on the tolerances and preferences defined in the user's mobility profile from Step 2.

Step 5: Optimal Path Computation (A* Search)
With a fully weighted, personalized graph, the final step is to find the best path. The **A* search algorithm** is executed on the graph. A* explores the paths from the start node, always prioritizing the path that has the lowest combined actual cost from the 'start' and estimated cost to the 'goal'. Since our edge costs are now a holistic representation of accessibility, comfort, and safety, A* will naturally find the route that is "best" for that specific user under the current real-world conditions, which is often, not the shortest one (geometrically). The output is the final, optimal sequence of path segments.

III. PROPOSED METHODOLOGY

A. Explanation about OSM and its parameters:
The digital blueprint of the physical world, OpenStreetMap (OSM) serves as giant collaborative mapping project. The fundamental pieces are nodes, ways, and tags.

Tag key	Example values	Meaning of Routing	Impact on cost
highway	footway, pedestrian, residential, primary, motorway	Path type classification	Determines P_{path} penalty multiplier
surface	asphalt, concrete, cobblestone, gravel, unpaved	Walking surface material	Affects $P_{weather}$ and $P_{surface}$ penalties
barrier	steps, kerb, bollard, gate	Physical obstacles	May block path or add penalties P_{slope}
wheelchair	yes, no, limited	Accessibility status	Critical for mobility-impaired users
incline	up, down, steep	Slope indication	Supplements DEM data for slope calculation P_{slope}
sidewalk	yes, no, separate, left, right	Sidewalk presence	Influences pedestrian routing preference P_{path}
foot	yes, no, designated	Foot traffic allowed	Determines if path is routable for pedestrians
access	yes, no, private, permissive	General access permissions	May exclude certain paths
max speed	30, 50, etc.	Speed limit (for roads)	Indicates traffic density/safety

Table 1. OSM tags and their meanings For Routing

Nodes: The basic element in OSM is **node**. It's a single point in space, defined by latitude and longitude.
In our equation a node is simply one of the numbered dots on the page. By itself, it's just a location. A node can represent anything that exists as a single point, such as a traffic intersection, a bus stop or a park bench, or a specific street address. Simply a point that helps define the curve of a road.

Ways: A **way** is what brings the map to life. It is a line or a shape consisting of an ordered list of nodes that are

connected. In our research, a way is the line used for connecting the dots in a specific sequence (e.g., from dot 1 to 2 to 3, and so on).
The term "path" referred, (like a pedestrian footpath or sidewalk) is represented in OSM as a specific *type* of way.

Tags: Tags are the most critical component for our research. They are the **descriptive labels** that give meaning to the nodes and ways. Without tags, OSM would just be a meaningless collection of points and lines. In simple terms, a tag consists of a key and a value. For example, a way with the tag highway = "footway" tells our system, "This line you are looking at is a sidewalk". If that same way also has the tag surface=asphalt, it adds the detail, "...and it's made of asphalt." A node with the tag "barrier steps" warns our system, "There is a flight of stairs at this specific point."

B. Equations

To accurately determine the true suitability of a path for a specific user, our methodology moves beyond simple distance. We calculate a holistic traversal cost, denoted as C(e), for each path segment in the graph computed as:

$$C(e)=L(e)*P_{risk}(e)*P_{weather}(e)*P_{path}(e)*P_{slope}(e)*P_{surface}(e)$$

This cost is computed using a dynamic, multiplicative penalty function, which quantifies the path's overall accessibility by scaling its physical length with a series of factors representing real-world safety, comfort, and user-defined preferences.

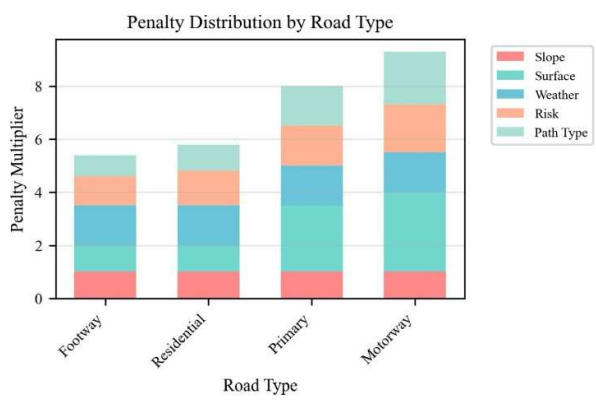

Fig 3. Distribution of penalty

C. Parameters:

Physical Length: This is the foundational, static component of our cost function. It represents the true geometric length of the edge in meters, as calculated from the geographic coordinates of its start and end nodes sourced from OpenStreetMap.

Predictive Risk Penalty: It measures the dynamic probability of encountering any obstacle. This risk is estimated using a Logistic Regression Model that is trained on a dataset of user-reported obstacles. This model uses parameters such as the current time of day and day of the week to predict the probability of an obstacle on edge e, denoted as $Prob_{ML}$. This is then converted into a penalty multiplier using the formula,

$$P_{risk}(e) = 1 + \alpha * Prob_{ML},$$

where alpha is a quantifiable hyperparameter that scales the penalty's severity.

Domain: $Prob_{ML}(e) \in [0,1]$ so $P_{risk}(e) \in [1, 1+\alpha]$.

$\alpha > 0$ and is recommended as default (e.g., $\alpha = 1.5$).

Weather Penalty : Its value is determined in real-time by querying a third-party weather API for the route's geographic location. The system then imposes a rule-based penalty; e.g., a "rain" condition might impose a multiplier of 1.5, which may be raised if the path surface is also labelled as "gravel". Clear conditions yield a base multiplier of 1.0.

Path Type Penalty : This penalty is derived from the static highway tag in OpenStreetMap and is used to favour pedestrian-dedicated infrastructure. The path segments are made cheaper than physical distance by tagging them as highway = "footway" or highway = "pedestrian" along with a beneficial multiplier (e.g., 0.8). Conversely, segments along busy roads tagged highway = "footway" receive a penalty multiplier greater than 1.0 to discourage their use.

Slope penalty (e): To calculate the slope, the system first queries a free Digital Elevation Model (DEM) API to retrieve the elevation of the edge's start node (u) and end node (v). The absolute slope is then calculated using the formula

$$slope = \frac{|Elev(v) - Elev(u)|}{L(e)}$$

$$P_{slope}(e:U) = 1, \quad \text{if slope}(e) \le U_{max}\ \text{slope}$$
$$= M_{impassable}, \quad \text{if slope}(e) > U_{max}\ \text{slope}$$
$$M_{impassable} = \infty \text{ (if we want to block that path)}$$

This value is compared against the maximum tolerable slope specified in the user's mobility profile (U_{max} slope). If the calculated slope exceeds this threshold, a significant penalty multiplier (e.g., 10.0) is applied to strongly discourage the path. Otherwise, the multiplier remains 1.0.

Surface Penalty : The user's personal comfort preferences is determined by multiplier. The surface material of the path is identified from its surface tag in OSM (e.g. asphalt, cobblestone, unpaved). The system then checks if this surface type is included in the user's list of disliked surfaces. If a match is found, a penalty multiplier is applied. Otherwise, the multiplier is 1.0.

D. Machine Learning Model for Risk Prediction

To estimate dynamic obstacle probabilities ($Prob_{ML}$)we trained a Logistic Regression model on a synthetic dataset of 50,000 timestamped samples simulating urban traffic patterns. The feature set included temporal attributes (Hour, Day_of_week) and contextual flags (Is_Rush_Hour, Is_Weekend) to predict binary obstacle presence. Training utilized an **80/20 train-test split** with class weighting to address the ~13.5% obstacle occurrence rate. The model achieved robust performance on the validation set, with an accuracy of **86.7%.**

E. The Admissible Heuristic Function

The A* algorithm uses a heuristic function (h(n)), which estimates the cost from a given node to the goal. The heuristic function must be admissible so that A* will find the best

(lowest-cost) path, and it should never overestimate the actual cost to the goal.

In a typical distance-based search, the Euclidean straight-line distance is an admissible and well-known heuristic. Our multiplicative cost model adds a special complication: since some penalties will have a value < 1.0 (e.g., for a desirable footway), the actual cost of a path can be less than its geometric distance. Hence, applying the basic Euclidean distance as the heuristic may cause an overestimation, violating the admissibility condition and possibly resulting in a suboptimal path. To solve this and guarantee admissibility, we define our heuristic h(n) as the Euclidean distance scaled by the lowest possible cost multiplier (n_{min}) present in our model.

The A* algorithm intelligently explores the graph to find the best path. It operates by evaluating each node it considers based on the famous A* formula:

$$f(n) = g(n) + h(n).$$

g(n): The actual cost of the path found so far from the start node to the current node n. This is calculated by summing the costs of all edges along this path.

h(n): The estimated cost from the current node n to the goal, as provided by our admissible heuristic function.

f(n): The total estimated cost of the best path that goes through node n.

A* prioritizes exploring paths with the lowest f(n) value. For each neighbour of the current node, the algorithm performs the following calculations to determine its potential as the next step in the optimal path:

1. Calculate Actual Cost (g(n)): First, it calculates the actual cost to get to this neighbour node along the current path. This is done by taking the known cost to reach the current node (g(current)) and adding the cost of the edge connecting them (C(current, neighbour)).

2. Calculate Estimated Future Cost (h(n)): Next, the algorithm calls our admissible heuristic function to calculate the estimated cost from this neighbour node to the final goal. This is the crucial step where h(n) is used.

$$h(n)=euclidean_distance(n,goal)*n_{min}$$

where,

$$n_{min} = \min(P_{risk}) * \min(P_{weather}) * \min(P_{path}) * \min(P_{slope}) * \min(P_{surface})$$

Based on our model, the minimum for P_{path} is **0.8** and the minimum for all other penalties is **1.0**

$$n_{min} = 1.0 * 1.0 * 0.8 * 1.0 * 1.0 = 0.8$$

By using n_{min}=0.8, our heuristic h(n) will always be an underestimation of the true cost. It calculates the "best-case scenario" (shortest distance on the most-preferred path type), guaranteeing that h(n) never overestimates the true path cost, thus preserving A*'s optimality.

3. Calculate Total Score (f(n)): The system then sums these two values to get the neighbour's total estimated cost: f(neighbour) =g(neighbour) + h(neighbour).

4. Update the Open List: The algorithm checks if this path to the neighbour is better (has a lower g(n)) than any path previously found. If it is, the neighbours information is updated, and it is added to the open list with its newly calculated f(n) score, making it a candidate for exploration in a future iteration.

E. Admissibility of the Heuristic Function

Let $h_{EU}(n)$be the Euclidean distance from node n to the goal. Let For every edge $e = (u, v)$, $L(e)$ = geometric length (meters).

n_{min} = min((P_{risk}) * min($P_{weather}$) * min(P_{path}) * min(P_{slope}) * min($P_{surface}$)). We define the heuristic as

$$h(n) = n_{\min} \cdot h_{EU}(n).$$

Then $h(n)$is admissible, i.e., it never overestimates the true cost.

Proof:

Let $n$be any node and let $P = (e_1, e_2, \dots, e_k)$be any path from $n$to the goal, where $e_i$are the edges in order. The cost of P is C(e)=L(e).p(e) be cost of edge.

$$Cost(P) = \sum C(e_i) = \sum L(e_i).p(e_i).$$

(Consider for ∑ i ranges from i=0 to i=K)

By definition of $n_{\min}$, for every $i$we have $p(e_i) \geq n_{\min}$. Therefore

$$Cost(P)=\sum L(e_i).\ p(e_i) \geq \sum L(e_i).\ n_{min}= n_{min}.\sum L(e_i)$$

By the triangle inequality (geometric length of path ≥straight-line distance), Therefore, for every possible path,

$$\sum L(e_i) \geq h_{EU}(n).$$

Combining the two inequalities,

$$Cost(P) \geq n_{min} * h_{EU}(n) = h(n).$$

$h^*(n)$= true optimal cost from $n$to the goal

The above holds for every path P from n to the goal, hence the minimal possible cost $h^*(n)$ satisfies $h^*(n) \geq h(n)$. Therefore $h(n) \leq h^*(n)$for all n, so $h$is admissible.

IV. Experimental Evaluation and Results

A. Experimental Setup

The experimental framework combined geospatial data (static) from OpenStreetMap(OSM) with environmental inputs (dynamic). Using Digital Elevation Model (DEM) API, different elevation profiles were derived to calculate the slope gradients. Real-time weather conditions were fetched through the OpenWeatherMap API. Additionally, a local SQLite database integrated user-reported obstacle data to assist the training of the predictive risk model. Evaluation was conducted using two distinct profiles: a "Wheelchair User" (defined by $U_{max_slope} < 7^o$ and a high penalty for surface barriers) and an "Elderly Citizen" (defined by $U_{max_slope} < 10^o$ and a moderate barrier penalty).

B. Quantitative Analysis

We compared the performance of our Dynamic Multi-Objective A* (DMOA*) algorithm against standard Dijkstra's algorithm and A* algorithm. We evaluated 10 distinct navigation queries across South Mumbai, including a

specific case study from CSMT to Marine Drive during peak hours.

Metric	Dijkstra's	A*	DMOA*
Avg. Route Length	2439 m	2601 m	2382 m
Avg. Accessibility Score	67.3	67.1	68.8
Avg. Execution Time	1850 ms	820 ms	1150 ms

Table 2. Algorithm Performance Comparison (Avg. over 10 trials)

As summarized in *Table 2*, the results show the efficiency of our proposed algorithm over the standard ones. While A* algorithm produced the longest route (2601m) with the lowest accessibility score (67.1), the DMOA* algorithm achieved the best performance across all metrics, with the shortest average route length (2382m) and the highest accessibility score (68.8). Although its execution time is higher than standard A* due to dynamic data fusion, it remains faster than the Dijkstra's algorithm. This indicates that it successfully identified efficient, user-friendly shortcuts that the standard algorithms likely ignored.

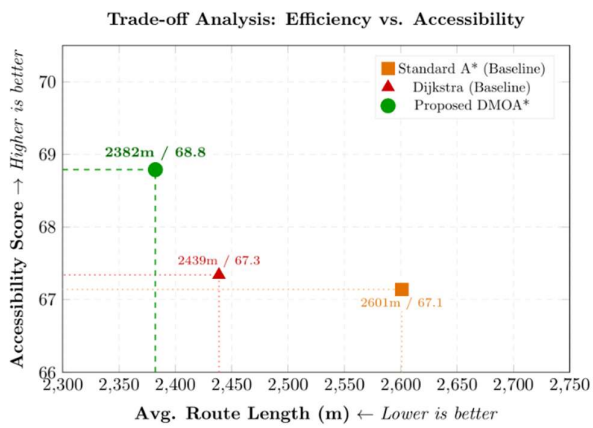

Fig 4. Statistical comparison of proposed DMOA* algorithm with baseline algorithms.

We analyze a critical edge from CSMT to Marine Drive for the "Wheelchair User" profile. This shows how DMOA* evaluates paths.

Standard Shortest Path (Dijkstra's route): A 150 m edge crossing a busy intersection with stairs:-

- L(e) = 150 m (Geometric Length)
- $P_{risk}(e) = 1.3$ (elevated risk due to traffic and crowds)
- $P_{weather}(e) = 1.0$ (clear weather)
- $P_{path}(e) = 1.8$ (major road crossing)
- $P_{slope}(e) = 10.0$ (impassable stairs for wheelchair)
- $P_{surface}(e) = 1.0$ (acceptable surface)
- **Final cost:** C(e) = 150 * 1.3 * 1.0 * 1.8 * 10.0 * 1.0 = **3510**

DMOA* Accessible Alternative: A 200 m detour using a signalized ramp crossing:

- L(e) = 200 m (Geometric Length)
- $P_{risk}(e) = 1.1$ (Moderate risk on dedicated path)
- $P_{weather}(e) = 1.0$ (clear weather)
- $P_{path}(e) = 0.8$ (pedestrian-friendly crossing)
- $P_{slope}(e) = 1.0$ (ramp, within tolerance)
- $P_{surface}(e) = 1.0$ (smooth asphalt)
- **Final cost:** C(e) = 200 * 1.1 * 1.0 * 0.8 * 1.0 * 1.0 = **176**

The cost difference (3510 vs 176) proves how DMOA* prioritizes accessibility over geometric distance, while not completely ignoring it, thereby selecting safer and highly navigable routes.

D. Case Study: CSMT to Marine Drive

Algorithm	Profile	Route Length	Accessibility Score	Est. Time
Dijkstra's	Wheelchair	2181.12m	65.7	30.3 min
Dijkstra's	Elderly	2181.12m	65.7	30.3 min
A*	Wheelchair	2821.93m	63.8	40.7 min
A*	Elderly	2821.93m	63.8	40.7 min
DMOA*	Wheelchair	1583.95m	71.4	21.7 min
DMOA*	Elderly	1719.98m	63.8	24.2 min

Table 3. CSMT to Marine Drive Case Study Results

We evaluated navigation query from Chhatrapati Shivaji Maharaj Terminus (CSMT) to Marine Drive during peak hours. As shown in *Table 3*, Dijkstra's provided a route of 2181.12 m with an accessibility score of 65.7. A* produced significantly longer route (2821.93m) with lower accessibility (63.8). DMOA* leveraged pedestrian-specific paths and identified routes that were both shorter and more accessible.

The DMOA* algorithm identified a route (1583m) significantly shorter than the Dijkstra baseline (2181m). This reduction is attributed to the beneficial path-type weighting P_{path}=0.8 assigned to dedicated footways. While standard algorithms defaulted to the connected road network, our system successfully utilized pedestrian-specific infrastructure such as footpaths and shortcuts restricted to vehicles demonstrating that accessibility-aware routing can simultaneously optimize for geometric efficiency.

E. Comparative System Analysis

The qualitative advantages of the proposed framework over the existing solutions have been highlighted in *Table 4*.

Aspect	Current Mapping Services	Proposed Multi-Objective Routing System
Routing Objective	Single objective: shortest geometric path	Multi-objective: accessibility, comfort, safety, user preferences
User Personalization	None - generic routing for all users	Personalized based on mobility profile and preferences

Dynamic Data Integration	Limited or none (static maps)	Real-time fusion of weather, risk, community reports
Accessibility Considerations	Basic wheelchair routing (if any)	Comprehensive: slope, surface, curb ramps, barriers
Data Sources	Proprietary map data	OpenStreetMap + external APIs + ML models + community data
Cost Function	Distance only	Multiplicative penalties: slope, surface, risk, weather, path type
Algorithm	Dijkstra/A* with distance heuristic	A* with admissible heuristic for penalty-based costs
Community Integration	None	User-reported obstacles and real-time updates
Weather Awareness	None	Weather-dependent penalties (e.g., slippery surfaces in rain)
Risk Assessment	None	ML-predicted obstacle probabilities based on time/day patterns

Table 4. Comparative Analysis: Current Mapping service vs Current Multi Objective routing system

While standard algorithms vary in performance, results show that they fail to provide safe routing for vulnerable groups. The proposed DMOA* system efficiently balances accessibility requirements with computational efficiency, providing an important improvement for mobility-impaired users.

V. CONCLUSION

The informational gap between the broad view of digital maps and the reality of the pedestrian environment is a major challenge of navigation in urban areas for people with mobility impairments. Through this paper, we presented a framework to bridge this gap by enhancing the A* algorithm using a multi-objective cost function. Our methodology is an integration of real-time community data, user preferences, and machine learning models to create a personalised routing experience. The case study results imply that our system can efficiently identify and generate accessible routes that may not be looked upon by traditional shortest path algorithms. Our multi-objective cost function offers the necessary intelligence for the A* algorithm to make user-centric decisions, by transforming abstract concepts like safety, comfort, and risk into penalties and benefits. This work promotes the concept for a new generation of navigation tools that give more importance to user well-being over distance optimization.

VI. ACKNOWLEDGEMENT

The authors Aayam Bajaj, Atharva Dhokte, and Yuvrajsing Bahure would like to extend their gratitude to the faculty members of the Information Technology Department at Bharati Vidyapeeth College of Engineering. Their guidance, support, and insights were beneficial in the successful execution of this research. We also wish to acknowledge the open-source community, whose work established the foundation for our project. We would also extend our gratitude to the contributors of OpenStreetMap for providing the rich geographical data and the developers of the Leaflet.js library for their efficient web mapping tools.

VII. REFERENCES

[1] D. Sanders et al., "Route Optimization using Forecasting, Wheelchair Modelling and Path Planning," *IEEE International Conference on Intelligent Transportation Systems, 2021.*

[2] Z. Wang et al., *"A route optimization model based on building semantics for pedestrian and wheelchair users,"* Automation in Construction, 2022.

[3] R. Doshmanziari et al., *"Experiment Design Considerations for Estimating Energy Expenditure During Wheelchair Propulsion,"* Heliyon, 2023.

[4] J. Darko et al., *"Adaptive personalized routing for vulnerable road users,"* IET Intelligent Transport Systems, 2022.

[5] R. van der Slikke et al., *"The Push Forward in Rehabilitation: Validation of a Machine Learning-Based Model for Detecting Passive or Active Wheelchair Use,"* Frontiers in Rehabilitation Sciences, 2024.

[6] R. Doshmanziari et al., *"Data-Driven Techniques for Estimating Energy Expenditure in Wheelchair Users,"* 2025.

[7] A. Cohen, A. Natapov, and S. Dalyot, "Leveraging OpenStreetMap to investigate urban accessibility and safety of visually impaired pedestrians," *Proceedings of the Academic Track at State of the Map 2022*, Florence, Italy, 2022.

[8] F. Saha et al., "MyPath: Accessible Routing for Wheelchair Users," *RESNA Annual Conference*, 2022.

[9] C. Prandi, L. Gkoumas, and F. S. N. Silva, "Semantic-enhanced personalized route planning for smart mobility," *I-CiTies 2025*, 2025.

[10] M. Ramezani et al., "Accessibility strategies to promote inclusive mobility through multi-objective approach," *SN Applied Sciences*, vol. 5, no. 150, 2023.